CS2810 Lab: The Optimized MagicalCrate

Template-Based Dynamic Storage System

Note : Use of STL is strictly prohibited. Memory leak should be properly managed. Last 8 test cases will fail if you don't manage memory properly.

Problem Overview

You are tasked with building a **MagicalCrate**, a smart container that automatically adjusts its storage capacity based on its contents. Unlike a standard array, this MagicalCrate grows when it reaches a threshold and shrinks when it reaches a threshold to save space.

Additionally, this MagicalCrate has a peculiar rule for removal: it always prioritizes removing the **"Heaviest" (Largest)** object currently inside to keep the load light!

This assignment requires you to implement:

1. **Global Template Functions** for searching and sorting.
2. A **Generic MagicalCrate Class** that manages dynamic memory and strictly follows specific resizing and removal rules.

Part 1: Global Template Functions

Implement the following global functions that work on any standard array. These will be used by both the testing interface and your MagicalCrate class.

1.1 Linear Search

```
template <typename T>
int linearSearch(T* arr, int size, T key);
```

Description: Returns the index of the first occurrence of `key`. Returns `-1` if not found.

1.2 Binary Search

```
template <typename T>
int binarySearch(T* arr, int size, T key);
```

Description: Performs a binary search on the array. Returns the index of the first occurence of key or `-1` if not found.

Note: You may assume the array is sorted before this function is called.

1.3 Sort

```
template <typename T>
void sort(T* arr, int size);
```

Description: Sorts the array in **ascending** order using any sorting algo. You are not allowed to use any STL, and implement it fully by yourself. (anything upto quadratic time complexity is fine.)

Part 2: The MagicalCrate Class

Implement a template class `MagicalCrate<T>` that manages a dynamic array with following requirements :

2.1 Data members

The class must maintain:

- `T* container`: A pointer to the dynamically allocated array.
- `int size`: The current number of elements stored in it.
- `int capacity`: The current maximum number of elements the array can hold.
- **Initial State:** A new MagicalCrate starts with `size = 0` and `capacity = 2`.

2.2 Dynamic Resizing Rules

- **Expansion:** Before adding an element, if `size == capacity/2`, you must **double** the capacity, (use the **new** keyword for dynamic memory allocation), and then insert the new element along with the already present elements in the new memory location.
- **Contraction:** After removing an element, if `size < (capacity / 4)` AND `capacity > 2`, you must **halve** the capacity.
- **Minimum Capacity:** The capacity should never drop below 2.

2.3 Required Methods

- `void add(T item)`: Adds an item to the MagicalCrate. Checks for expansion *before* adding.

- `void remove()`: **T**his function finds and removes the first occurence of **heaviest (largest)** element in the MagicalCrate and prints it. After removing it, if there are elements on right of it, they all must get shifted to its left by 1 position, so that there is no gap between the elements.
  - If the MagicalCrate is empty, print `"Error: Crate is empty"`.
- `int find(T item)`: Prints the index of the item using your global `linearSearch`.
- `void printSize()`: Prints `"Size: <size>"`.
- `void printCapacity()`: Prints `"Capacity: <capacity>"`.
- `void print()`: Prints the elements in a single line separated by a space in the order in which they are present.

Input Format

Mode 1: Global Testing

Tests the global search and sort functions on a raw array.

- **Line 1:** `"global"`
- **Line 2:** Data type (`int` or `double`)
- **Line 3:** `N` (size of array)
- **Line 4:** `N` space-separated elements
- **Line 5:** `K` (number of operations)
- **Next K lines:** valid Operations are as follows :
  - `linearSearch <val>` : print index of val or -1 if not found.
  - `binarySearch <val>` : print index of val or -1 if not found.
  - `sort` : sorts the array, and prints "Sorted".

Mode 2: MagicalCrate Testing

Tests the dynamic MagicalCrate class that you have implemented.
Note: The keyword "global" is absent here

- **Line 1:** Data type (`int` or `double`)
- **Next lines:** A series of commands until the end of input, possible commands :

- `add <val>` : calls add(val).
- `remove` : calls remove().
- `find <val>` : calls find(val).
- `size` : calls printSize().
- `capacity` : calls printCapacity().
- `print` : calls print().

Sample Input / Output

Output of each operation should be printed on a different line as shown in sample output.

Sample 1: Global Mode

Input:

```
global
int
5
50 10 40 20 30
3
linearSearch 20
sort
binarySearch 40
```

Output:

```
3
Sorted
3
```

Sample 2: Crate Mode

Input:

```
double
add 10.5
add 5.5
add 20.5
size
capacity
print
```

```
remove
print
capacity
```

Output:

```
Size: 3
Capacity: 8
10.5 5.5 20.5
20.5
10.5 5.5
Capacity: 8
```

Constraints

- The number of elements in the array or MagicalCrate will be ≤ 100 at any point in time.
- The number of operations K will be ≤ 20.
- Element values will be within the range of standard `int` and `double`.

Test Cases Breakdown

Your code will be tested against a total of **60 test cases**. The breakdown of these tests is as follows:

Test Case IDs	Description
1 – 4	Global Linear Search on raw array
5 – 8	Global Binary Search on raw array
9 – 12	Global Sorting on raw array
13 – 20	Global Mixed Operations on raw array
21 – 52	MagicalCrate Operations (Add, Remove, Resize, etc.)
53 – 60	MagicalCrate Operations with **Memory Leak Check**

Note on Memory Leak Checks (Tests 53–60):
These final 8 test cases involve intensive add/remove operations. They are run with a strict memory limit and/or Valgrind. If your `MagicalCrate` class fails to properly `delete[]` old arrays during resizing or destruction, you will fail these tests.